

# **DEPARTMENT OF MECHANICAL ENGINEERING**

## **MECHATRONICS & IOT 24MEC51**

### **EXERCISE NO: 12**

|                         |                   |
|-------------------------|-------------------|
| <b>GURUTHARSAN S</b>    | <b>- 24MER011</b> |
| <b>HARISH KUMAR K K</b> | <b>- 24MER015</b> |
| <b>MADHANRAJ K</b>      | <b>- 24MER028</b> |
| <b>SADAAGOPAAN T R</b>  | <b>- 24MER052</b> |

# MECHATRONICS & IoT

## ASSIGNMENT REPORT – 12

### **TITLE:**

Design and develop an Automated Color-Based Product Sorting System for Manufacturing Industries. The system automatically detects and sorts products based on color for industrial automation applications.

### **Done By:**

GURTHARSAN S - 24MER011

HARISH KUMAR K K – 24MER015

MADHANRAJ K – 24MER028

SADAAGOPAAN T R – 24MER052

## **Project Overview:**

The Automated Color-Based Product Sorting System is designed to automatically identify and sort products according to their color. The system uses a TCS3200 color sensor to detect the color of an object and an ESP8266 NodeMCU as the main controller.

A servo motor is used as the sorting mechanism. When a product passes in front of the color sensor, the sensor detects its color and sends the information to the ESP8266. The controller processes the detected color and rotates the servo motor to direct the product into the appropriate sorting section.

The system demonstrates the application of industrial automation, sensor-based inspection, and automatic material handling.

## **Problem Statement**

In many manufacturing industries, products need to be separated according to color before packaging, assembly, or further processing. Manual color-based sorting can be:

- Slow
- Labour-intensive
- Inconsistent
- Prone to human error
- Difficult to operate continuously
- Less suitable for high-speed production

Therefore, an automated color-based sorting system is required to identify products accurately and automatically separate them according to their color.

.

## Proposed Solution

The proposed system uses a TCS3200 color **sensor** to identify the color of products moving through the sorting area.

The ESP8266 NodeMCU receives the sensor output and determines the product color.

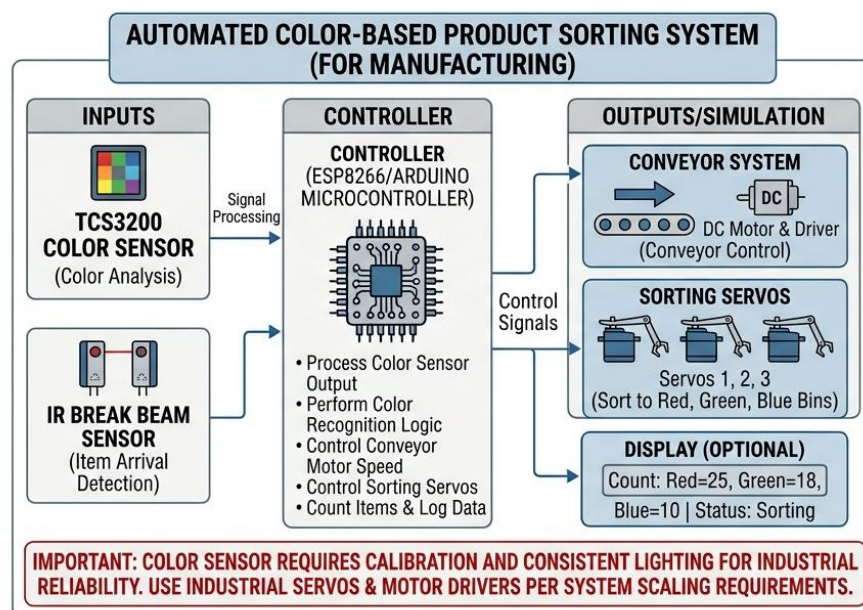
A servo motor acts as the sorting mechanism.

Example:

RED PRODUCT → SERVO POSITION 1 → RED BIN  
GREEN PRODUCT → SERVO POSITION 2 → GREEN BIN  
BLUE PRODUCT → SERVO POSITION 3 → BLUE BIN

LED indicators can be used to show the detected color.

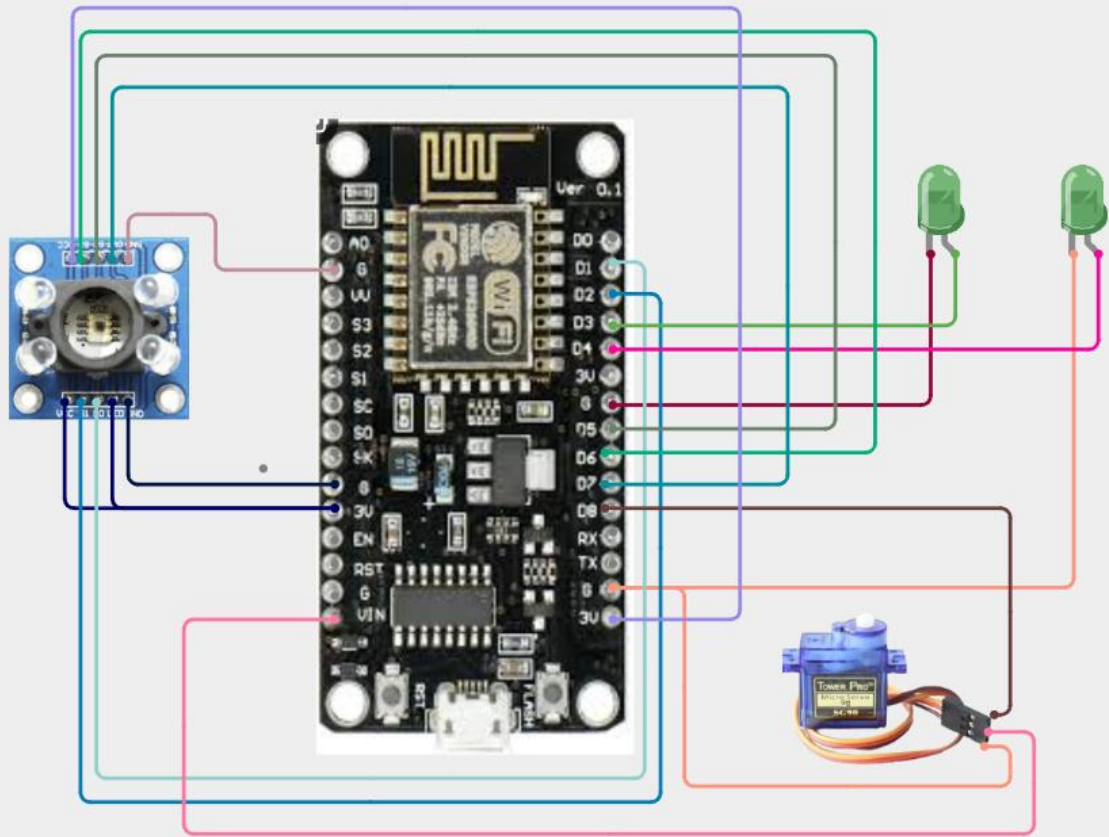
## System Architecture



## Circuit Table

| S.No. | Component   | Pin      | ESP8266 Connection                 |
|-------|-------------|----------|------------------------------------|
| 1     | TCS3200     | VCC      | Suitable supply                    |
| 2     | TCS3200     | GND      | GND                                |
| 3     | TCS3200     | S0       | D1                                 |
| 4     | TCS3200     | S1       | D2                                 |
| 5     | TCS3200     | S2       | D5                                 |
| 6     | TCS3200     | S3       | D6                                 |
| 7     | TCS3200     | OUT      | D7                                 |
| 8     | Servo Motor | Signal   | D4                                 |
| 9     | Servo Motor | VCC      | External 5 V supply                |
| 10    | Servo Motor | GND      | Common GND                         |
| 11    | Red LED     | Anode    | D0 through 220 $\Omega$            |
| 12    | Red LED     | Cathode  | GND                                |
| 13    | Green LED   | Anode    | D3 through 220 $\Omega$            |
| 14    | Green LED   | Cathode  | GND                                |
| 15    | Blue LED    | Anode    | Suitable GPIO through 220 $\Omega$ |
| 16    | Blue LED    | Cathode  | GND                                |
| 17    | Buzzer      | Positive | Suitable GPIO                      |
| 18    | Buzzer      | Negative | GND                                |

## Circuit Design



## Algorithm

1. START
2. Initialize ESP8266 NodeMCU.
3. Initialize TCS3200 color sensor.
4. Initialize servo motor.
5. Initialize LED indicators and buzzer.
6. Set the initial servo position.
7. Wait for a product to be placed  
in front of the color sensor.
8. Read the RED component.
9. Read the GREEN component.
10. Read the BLUE component.
11. Compare the RGB values.
12. Identify the product color.
13. If RED is detected:  
    Turn ON Red LED.  
    Move servo to Red position.  
    Direct product to Red bin.
14. Else if GREEN is detected:  
    Turn ON Green LED.  
    Move servo to Green position.  
    Direct product to Green bin.
15. Else if BLUE is detected:  
    Turn ON Blue LED.

Move servo to Blue position.  
Direct product to Blue bin.

16. Else:

Turn ON buzzer.  
Identify product as UNKNOWN.  
Send product to reject/unknown bin.

17. Return servo to its initial position.

18. Wait for the next product.

19. Repeat the process continuously.

20. STOP.

## Coding

```
#include <Servo.h>
```

```
#include <ESP8266WiFi.h>
```

```
#include "AdafruitIO_WiFi.h"
```

```
//===== WiFi =====
```

```
#define WIFI_SSID "Harish Kumar K K"
```

```
#define WIFI_PASS "Mavboiii@10"
```

```
//===== Adafruit IO =====
```

```
#define IO_USERNAME "Jr_Frost"
```

```
#define IO_KEY "aio_obVD87ub3BbEG4osjIOQjHNrUn7m"
```



```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
```

```
// Adafruit Feeds
```

```
AdafruitIO_Feed *redFeed = io.feed("red-value");
```

```
AdafruitIO_Feed *greenFeed = io.feed("green-value");
```

```
AdafruitIO_Feed *blueFeed = io.feed("blue-value");
```

```
AdafruitIO_Feed *colorFeed = io.feed("detected-color");
```

```
//===== TCS3200 =====
```

```
#define S0_PIN D1
```

```
#define S1_PIN D2
```

```
#define S2_PIN D3
```

```
#define S3_PIN D4
```

```
#define SENSOR_OUT D6
```

```
//===== Servo =====
```

```
#define SERVO_PIN D5
```

```
//===== LEDs =====
```

```
#define RED_LED D7
```

```
#define GREEN_LED D0
```

```
#define BLUE_LED D8
```

```
//===== Servo Angles =====
```

```
#define HOME_ANGLE 90
```

```
#define RED_ANGLE 30
```

```
#define GREEN_ANGLE 90
```

```
#define BLUE_ANGLE 150
```

```
Servo sortingServo;
```

```
//=====
```

```
// Setup
```

```
//=====
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    pinMode(S0_PIN, OUTPUT);
```

```
    pinMode(S1_PIN, OUTPUT);
```

```
    pinMode(S2_PIN, OUTPUT);
```

```
pinMode(S3_PIN, OUTPUT);
```

```
pinMode(SENSOR_OUT, INPUT);
```

```
digitalWrite(S0_PIN, HIGH);
```

```
digitalWrite(S1_PIN, LOW);
```

```
pinMode(RED_LED, OUTPUT);
```

```
pinMode(GREEN_LED, OUTPUT);
```

```
pinMode(BLUE_LED, OUTPUT);
```

```
turnOffLEDs();
```

```
sortingServo.attach(SERVO_PIN);
```

```
sortingServo.write(HOME_ANGLE);
```

```
Serial.println("Connecting to Adafruit IO...");
```

```
io.connect();
```

```
while (io.status() < AIO_CONNECTED) {
```

```
    Serial.print(".");
```

```
    delay(500);
```

```
}
```

```
Serial.println();
```

```
Serial.println("Connected to Adafruit IO!");
```

```
Serial.println(io.statusText());
```

```
delay(1000);
```

```
Serial.println("System Ready");
```

```
}
```

```
//=====
```

```
void turnOffLEDs() {
```

```
    digitalWrite(RED_LED, LOW);
```

```
    digitalWrite(GREEN_LED, LOW);
```

```
    digitalWrite(BLUE_LED, LOW);
```

```
}
```

```
//=====
```

```
unsigned long readSensor(bool s2, bool s3) {
```

```
    digitalWrite(S2_PIN, s2);
```

```
    digitalWrite(S3_PIN, s3);
```

```
    delay(30);
```

```
    return pulseIn(SENSOR_OUT, LOW, 100000);
```

```
}
```

```
unsigned long readRed() {
```

```
    return readSensor(LOW, LOW);
```

```
}
```

```
unsigned long readGreen() {
```

```
    return readSensor(HIGH, HIGH);
```

```
}
```

```
unsigned long readBlue() {
```

```
    return readSensor(LOW, HIGH);  
}
```

```
//=====
```

```
void uploadValues(unsigned long r, unsigned long g, unsigned long b) {
```

```
    redFeed->save((int)r);
```

```
    greenFeed->save((int)g);
```

```
    blueFeed->save((int)b);
```

```
}
```

```
//=====
```

```
void sortRed() {
```

```
    Serial.println("RED DETECTED");
```

```
    colorFeed->save("RED");
```

```
turnOffLEDs();
```

```
digitalWrite(RED_LED, HIGH);
```

```
sortingServo.write(RED_ANGLE);
```

```
delay(1500);
```

```
sortingServo.write(HOME_ANGLE);
```

```
delay(500);
```

```
digitalWrite(RED_LED, LOW);
```

```
}
```

```
//=====
```

```
void sortGreen() {
```

```
Serial.println("GREEN DETECTED");
```

```
colorFeed->save("GREEN");
```

```
turnOffLEDs();
```

```
digitalWrite(GREEN_LED, HIGH);
```

```
sortingServo.write(GREEN_ANGLE);
```

```
delay(1500);
```

```
sortingServo.write(HOME_ANGLE);
```

```
delay(500);
```

```
digitalWrite(GREEN_LED, LOW);
```

```
}
```

```
//=====
```

```
void sortBlue() {
```

```
Serial.println("BLUE DETECTED");
```

```
colorFeed->save("BLUE");
```



```
turnOffLEDs();
```

```
digitalWrite(BLUE_LED, HIGH);
```

```
sortingServo.write(BLUE_ANGLE);
```

```
delay(1500);
```

```
sortingServo.write(HOME_ANGLE);
```

```
delay(500);
```

```
digitalWrite(BLUE_LED, LOW);
```

```
}
```

```
//=====
```

```
void unknownColor() {
```

```
Serial.println("UNKNOWN COLOR");
```

```
colorFeed->save("UNKNOWN");
```

```
turnOffLEDs();
```

```
sortingServo.write(HOME_ANGLE);
```

```
delay(500);
```

```
}
```

```
// Main Loop
```

```
void loop() {
```

```
  io.run(); // Keep Adafruit connection alive
```

```
  unsigned long red;
```

```
  unsigned long green;
```

```
  unsigned long blue;
```

```
  Serial.println();
```

```
  Serial.println("Reading color...");
```

```
  red = readRed();
```

```
  delay(20);
```

```
green = readGreen();
```

```
delay(20);
```

```
blue = readBlue();
```

```
Serial.print("RED  = ");
```

```
Serial.println(red);
```

```
Serial.print("GREEN = ");
```

```
Serial.println(green);
```

```
Serial.print("BLUE  = ");
```

```
Serial.println(blue);
```

```
uploadValues(red, green, blue);
```

```
if (red == 0 || green == 0 || blue == 0) {
```

```
    Serial.println("Invalid reading");
```

```
    delay(500);

    return;
}

if ((red < green * 0.75) && (red < blue * 0.75)) {

    sortRed();

}

else if ((green < red * 0.80) && (green < blue * 0.80)) {

    sortGreen();

}

else if ((blue < red * 0.75) && (blue < green * 0.75)) {

    sortBlue();

}

else {
```

```
    unknownColor();  
}  
  
Serial.println("-----");  
  
delay(1000);  
}
```

## **System Working**

### **Step 1 – Product Detection**

A product is placed in front of the TCS3200 sensor.

The sensor detects the reflected light from the product.

### **Step 2 – Color Measurement**

The TCS3200 measures the RGB components of the reflected light.

The ESP8266 reads the sensor's output frequency.

### **Step 3 – Color Identification**

The controller compares the RGB values.

For example:

Red value > Green value

Red value > Blue value

↓

RED PRODUCT

#### Step 4 – Sorting Decision

After identifying the color, the ESP8266 sends a control signal to the servo motor.

#### Step 5 – Product Sorting

The servo moves the sorting arm to the appropriate position.

Red → Red Bin

Green → Green Bin

Blue → Blue Bin

#### Step 6 – Servo Reset

After sorting, the servo returns to its initial position and waits for the next product.

## Bills Of Material

| S.N<br>o. | Component                   | Specification            | Quantity | Purpose               |
|-----------|-----------------------------|--------------------------|----------|-----------------------|
| 1         | ESP8266<br>NodeMCU          | Wi-Fi<br>Microcontroller | 1        | Main Controller       |
| 2         | TCS3200<br>Color<br>Sensor  | RGB Color<br>Sensor      | 1        | Color Detection       |
| 3         | Servo<br>Motor              | SG90                     | 1        | Sorting<br>Mechanism  |
| 4         | Red LED                     | 5 mm                     | 1        | Red Indication        |
| 5         | Green<br>LED                | 5 mm                     | 1        | Green Indication      |
| 6         | Blue LED                    | 5 mm                     | 1        | Blue Indication       |
| 7         | Resistor                    | 220 $\Omega$             | 3        | LED Protection        |
| 8         | Buzzer                      | 3.3/5 V                  | 1        | Alert                 |
| 9         | Breadboard                  | Standard                 | 1        | Prototype<br>Assembly |
| 10        | Jumper<br>Wires             | Male-Male                | 1 Set    | Connections           |
| 11        | USB<br>Cable                | Micro USB                | 1        | Programming/Power     |
| 12        | External<br>Power<br>Supply | Suitable 5 V             | 1        | Servo Power           |

## Prototype Implementation

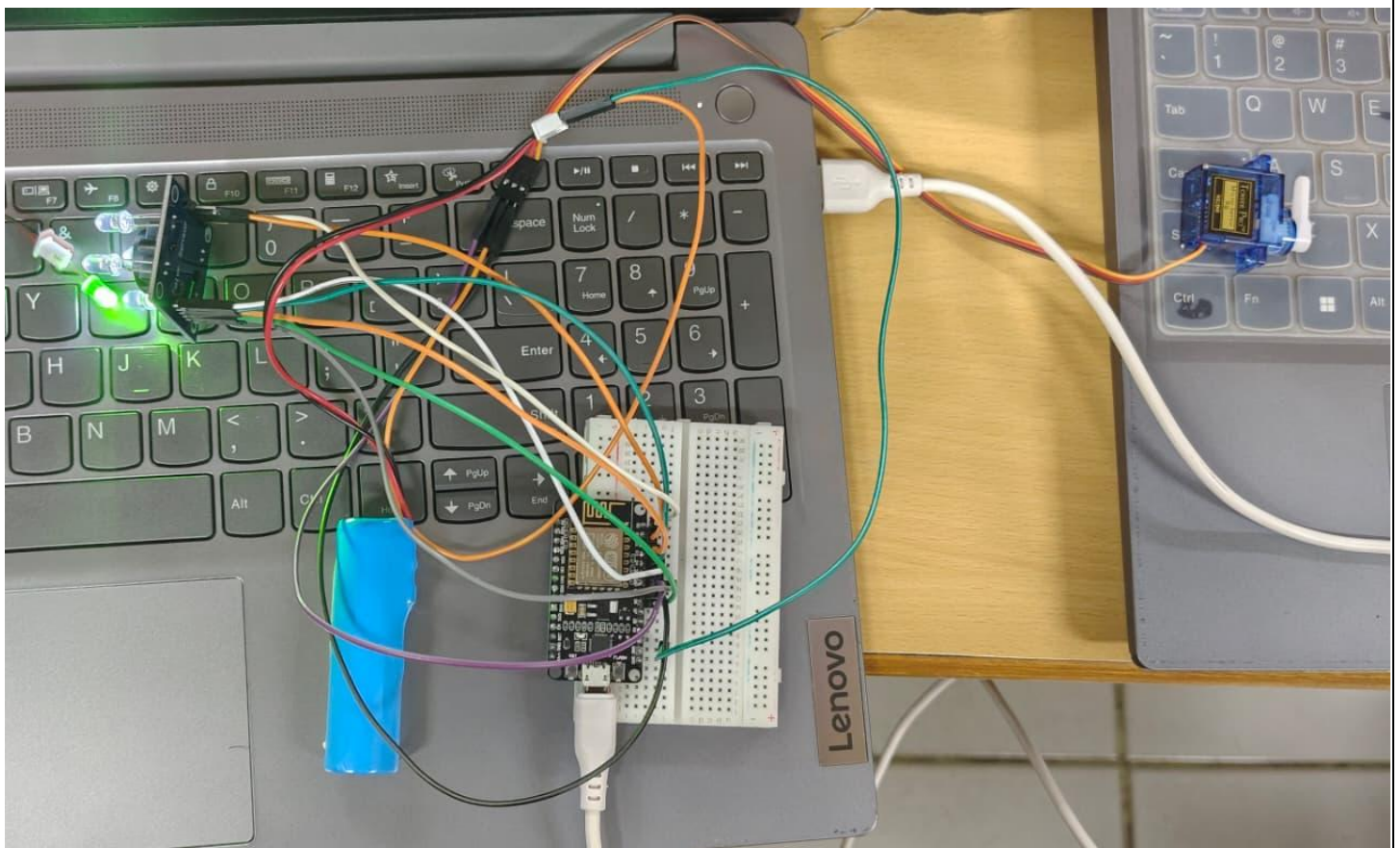
The prototype consists of an ESP8266 NodeMCU, TCS3200 color sensor, servo motor, LEDs, buzzer, breadboard, and jumper wires.

The TCS3200 is positioned above or beside the product path. When a product reaches the sensing position, the sensor detects its color.

The ESP8266 processes the sensor readings and identifies the product. The servo motor then moves the sorting mechanism to the corresponding position.

After the product is sorted, the servo returns to its neutral position and the system waits for the next product.

The prototype can be extended by adding a DC motor and conveyor belt to create a continuous industrial sorting line.





## Results & Performance Analysis

The prototype successfully demonstrates automatic color identification and product sorting.

| Parameter               | Result    |
|-------------------------|-----------|
| Product Color Detection | Yes       |
| Red Detection           | Yes       |
| Green Detection         | Yes       |
| Blue Detection          | Yes       |
| Automatic Sorting       | Yes       |
| Servo Control           | Yes       |
| Visual Indication       | Yes       |
| Unknown Color Detection | Supported |
| Continuous Operation    | Supported |

### Performance Analysis

The system reduces the need for manual product sorting and provides consistent sorting based on the programmed color thresholds.

Performance can be improved by:

- Calibrating the TCS3200 sensor.
- Maintaining constant illumination.
- Keeping the sensor at a fixed distance.
- Using a controlled conveyor speed.
- Filtering noisy sensor readings.
- Using multiple readings before making a sorting decision.

THROTTLE WARNING: madhan\_2006 data rate limit reached, 12 seconds until throttle released



madhan\_2006 / Dashboards / automatic color based product sorting system

+ New Block

Edit Layout



481

431

323

RED

```
BLUE = 468
RED DETECTED
-----
```

```
Reading color...
RED = 300
GREEN = 411
BLUE = 447
RED DETECTED
-----
```

```
Reading color...
RED = 290
GREEN = 313
BLUE = 240
UNKNOWN COLOR
-----
```

```
Reading color...
RED = 295
GREEN = 262
BLUE = 155
BLUE DETECTED
-----
```

```
Reading color...
RED = 279
GREEN = 258
BLUE = 158
BLUE DETECTED
-----
```

```
Reading color...
RED = 277
GREEN = 268
BLUE = 203
UNKNOWN COLOR
-----
```

```
Reading color...
RED = 282
GREEN = 282
BLUE = 189
BLUE DETECTED
-----
```

```
Reading color...
RED = 295
GREEN = 253
BLUE = 152
BLUE DETECTED
-----
```

Reading color...

RED = 93

GREEN = 49

BLUE = 106

GREEN DETECTED

Reading color...

RED = 313

GREEN = 425

BLUE = 468

RED DETECTED

RED = 462  
GREEN = 562  
BLUE = 607  
UNKNOWN COLOR

Reading color...

RED = 296  
GREEN = 406  
BLUE = 474  
RED DETECTED

Reading color...

RED = 319  
GREEN = 447  
BLUE = 500  
RED DETECTED

Reading color...

RED = 312  
GREEN = 406  
BLUE = 465  
UNKNOWN COLOR

Reading color...

RED = 182  
GREEN = 120  
BLUE = 197  
GREEN DETECTED

Reading color...

RED = 120  
GREEN = 72  
BLUE = 148  
GREEN DETECTED

Reading color...

RED = 96  
GREEN = 49  
BLUE = 111  
GREEN DETECTED

Reading color...

RED = 85  
GREEN = 45  
BLUE = 102  
GREEN DETECTED